

Retrieval-Augmented Generation (RAG)

Pipeline, chunking, retrieval techniques & interview essentials

1. What is RAG (Retrieval-Augmented Generation)?

- A technique that grounds an LLM's response in relevant external documents retrieved at query time, instead of relying only on knowledge baked into its weights.
- Combines an **information retrieval** system (finds relevant text) with a **generator** (the LLM, which writes the answer using that text as context).
- Solves three core LLM problems: **knowledge cutoff** (data can be updated without retraining), **hallucination** (answers are grounded in real sources), and **private/domain data** (LLM can answer from your own docs without fine-tuning).

2. RAG Pipeline — Two Phases

- **A. Indexing (offline, done once / on updates):** Load documents → split into chunks → embed each chunk into a vector → store vectors + metadata in a vector database.
- **B. Retrieval + Generation (online, per query):** Embed the user's query with the same embedding model → search the vector store for the most similar chunks → insert those chunks into the LLM prompt as context → LLM generates an answer grounded in that context.
- The augmented prompt typically looks like: *"Using only the context below, answer the question. Context: {retrieved chunks}. Question: {user query}"* — this instructs the model to stay grounded and reduces hallucination.

3. Chunking Strategies

- **Fixed-size chunking:** Split every N tokens/characters, usually with overlap (e.g. 500 tokens, 50 overlap) so context isn't cut mid-idea. Simple but can split sentences/ideas awkwardly.
- **Recursive/structure-aware chunking:** Split along natural boundaries first (sections → paragraphs → sentences) before falling back to fixed size — preserves semantic units better.
- **Semantic chunking:** Group sentences by embedding similarity so each chunk is topically coherent, rather than by a fixed length.
- **Document-aware chunking:** Respect structure like markdown headers, tables, or code blocks so a chunk doesn't split a table row or function mid-way.
- Chunk size is a trade-off: too small → loses context; too large → dilutes relevance and wastes context-window budget on noise.

4. Embedding Models & Vector Databases

Vector DB	Type	Notable Trait
FAISS	Library	Facebook's fast in-memory similarity search library; not a managed DB.
Chroma	Open-source	Lightweight, easy local setup, popular for prototyping.
Pinecone	Managed/cloud	Fully managed, scales easily, popular in production.
Weaviate	Open-source/cloud	Built-in hybrid search and modules for embeddings.
Milvus	Open-source	Built for large-scale, high-performance vector search.
pgvector	Postgres extension	Adds vector search to an existing Postgres database.

- Embedding models (e.g. OpenAI text-embedding-3, Cohere Embed, BGE, E5, Sentence-Transformers) convert text into fixed-size vectors so semantic similarity becomes a geometric distance (cosine similarity / dot product) between vectors.

5. Retrieval Techniques

- **Dense/similarity search:** Cosine similarity or dot product between query and chunk embeddings — captures semantic meaning, not just keyword overlap.
- **Sparse/keyword search (BM25):** Classic term-frequency-based search — strong for exact terms, IDs, acronyms that embeddings can miss.
- **Hybrid search:** Combines dense + sparse (BM25) results, often merged via reciprocal rank fusion — usually outperforms either alone.
- **Re-ranking:** A second, more expensive cross-encoder model re-scores the top-k retrieved chunks for finer relevance ranking before they reach the LLM.
- **Metadata filtering:** Narrow the search using structured filters (date, source, user permissions) alongside vector similarity.

6. Advanced RAG Techniques

- **Query rewriting/expansion:** Use an LLM to reformulate a vague or multi-part user query into one or more better search queries.
- **HyDE (Hypothetical Document Embeddings):** Ask the LLM to first draft a hypothetical answer, then embed and search with that — often matches real documents better than the raw question.
- **Multi-query retrieval:** Generate several query variations, retrieve for each, then merge/de-duplicate results for broader recall.

- **Parent-document retriever:** Search over small child chunks for precision, but return the larger parent chunk/section to the LLM for fuller context.
- **Graph RAG:** Build a knowledge graph of entities/relations from documents and retrieve via graph traversal — better for multi-hop, relational questions than flat vector search.
- **Agentic RAG:** Let the LLM decide iteratively whether to retrieve again, rewrite the query, or answer — used when a single retrieval pass isn't enough.

7. Evaluating a RAG System

- **Retrieval metrics:** Precision@k / Recall@k — did the right chunks get retrieved at all?
- **Faithfulness / groundedness:** Is the generated answer actually supported by the retrieved context, or did the model add unsupported claims?
- **Answer relevance:** Does the final answer actually address the user's question?
- **Context precision/recall:** Of the retrieved chunks, how many were actually useful (precision) and how much of the needed info was captured (recall)?
- Common tooling: RAGAS, TruLens, and LLM-as-judge pipelines that score faithfulness/relevance automatically.

8. Challenges & Failure Modes

- **Bad chunking** splits relevant info across chunks so no single chunk is a good match.
- **Retrieval miss:** The right document exists but isn't retrieved (embedding model mismatch, poor query phrasing).
- **"Lost in the middle":** LLMs can under-use information placed in the middle of a long context window.
- **Stale index:** Vector store not updated when source documents change.
- **Irrelevant context injection:** Retrieved noise can distract the model or dilute the truly relevant chunk.
- **Latency/cost stacking:** Embedding + search + re-ranking + generation all add up in a multi-step pipeline.

9. Quick-Fire Interview Answers

- **RAG vs. fine-tuning?** RAG injects fresh/external knowledge at inference time without retraining; fine-tuning bakes in style/behavior/domain patterns into the weights. They're complementary, not mutually exclusive.
- **Why not just put everything in the context window?** Cost, latency, the "lost in the middle" effect, and many knowledge bases exceed even large context windows — retrieval selects only what's relevant.
- **How do you pick chunk size?** Balance semantic coherence against retrieval precision; typically 200–800 tokens with slight overlap, tuned empirically per document type.
- **Why hybrid search over pure vector search?** Vector search can miss exact matches (IDs, rare terms, acronyms) that keyword/BM25 search catches — combining both improves recall.
- **One-line summary:** RAG turns an LLM from a closed-book exam-taker into an open-book one — it retrieves relevant, current, or private knowledge and grounds the model's answer in it.